AI FAQ ASSISTANT

Naan Mudhalvan – Final Year Assessment Project Documentation

TEAM DETAILS

Role

Name

Team Leader

Ezhilan E

Team Member

Balaji S

Team Member

Abdul Azeem S

Team Member

Ganesh M

1. Abstract

The AI FAQ Assistant is an AI-powered FAQ and customer-support automation system. It provides a secure REST API for creating, managing, searching and generating Frequently Asked Questions. The system uses Node.js and Express.js for the backend, MongoDB with Mongoose for data storage and Google Gemini AI for automated FAQ generation. JWT authentication and bcrypt password hashing are used to protect user accounts and private operations.

2. Introduction

Customer-support teams often handle many repeated questions. Creating and maintaining FAQ content manually can take time and make information difficult to organize. The AI FAQ Assistant provides a centralized backend that helps users create FAQs, organize them into categories, search for answers and generate FAQ content using AI.

3. Problem Statement

- Manual preparation of support questions and answers is time-consuming.
- Maintaining updated FAQ documentation is difficult.
- Finding relevant information quickly can be challenging.

- Private FAQ records need proper access control.
- Support teams need a simple way to generate and organize FAQ content.

4. Objectives

- Provide a centralized FAQ management system.
- Allow authenticated users to create, update and delete their own FAQs.
- Generate FAQ questions and answers using Google Gemini AI.
- Provide keyword/semantic FAQ search.
- Organize FAQs using categories.
- Protect private operations using JWT authentication.
- Store data safely using MongoDB and Mongoose validation.
- Provide consistent error handling through the backend.

5. Proposed Solution

The system provides a secure REST API that separates business logic from the user interface. Authentication controls protected operations, Mongoose manages database schemas, and AI services generate FAQ content.

Public users can search and view published FAQs, while authenticated users can manage their permitted records.

6. Software Requirements

- Operating System: Windows 10/11, macOS or Linux
- Node.js: Version 18 or above
- npm: Version 9 or above
- Express.js: Version 4/5 ecosystem
- MongoDB and Mongoose ODM
- Postman or ThunderClient for API testing
- Visual Studio Code or another code editor
- Google Gemini SDK / AI service

7. Hardware Requirements

- Processor: Intel Core i5 8th Gen or above / AMD Ryzen 5 or equivalent
- RAM: Minimum 8 GB; 16 GB recommended
- Storage: At least 1 GB available workspace

8. System Architecture

The application follows a layered backend architecture.

1. **API Interface – tools such as Postman send requests and receive JSON responses.**

2. **Express Server – receives HTTP requests such as GET, POST, PUT and DELETE.**

3. **Authentication Middleware – validates JWT tokens and protects private routes.**

4. **Service/Logic Layer – handles application rules and operations.**

5. **Database Layer – Mongoose communicates with MongoDB.**

6. **AI Integration – Google Gemini is used for question-answer generation and topic generation.**

9. Database Design

The system uses MongoDB collections with logical relationships.

User Entity:

- _id
- name
- email
- password (hashed)
- role
- createdAt
- updatedAt

FAQ Entity:

- _id
- question ¬ؤ answer
- category
- createdBy
- createdAt
- updatedAt

AI Generation Entity:

- topic
- generatedQuestion

- generatedAnswer
- generatedCategory ¬ generatedAt

Category Entity:

- categoryName
- description ¬ totalFAQs

10. Relationships

- User → FAQ: One user can create and manage many FAQ records.
- Category → FAQ: One category can contain many FAQ records.
- User → AI Generated FAQ: A user can generate multiple AI-assisted FAQs.
- FAQ → Search Results: Search operations retrieve matching FAQ documents using keyword and semantic matching.

11. Key Features

- JWT Authentication and Authorization
- AI-powered FAQ Generation
- Semantic/keyword FAQ Search
- Create, Read, Update and Delete (CRUD) operations
- FAQ Category Management
- Password Hashing using bcrypt
- Mongoose Schema Validation
- Centralized Error Handling
- RESTful API Architecture
- MongoDB Integration

12. User Roles

Admin:

- Manage registered users.
- View, update and delete any FAQ.
- Monitor AI-generated content.
- Manage categories.
- Access complete system functionality.

Content Creator:

- Create FAQs.
- Edit and delete personal FAQs.
- Generate FAQs using AI.
- Organize FAQs by category.

Authenticated User:

- View public FAQs.
- Search FAQs.
- Generate AI-assisted answers.
- Access personal profile information.

Public User:

- View published FAQs.
- Search FAQs.
- Cannot create, edit or delete FAQ records.
- Cannot access protected API endpoints.

13. MVC Architecture

The backend follows the MVC concept.

Model Layer: Defines Mongoose schemas and database structures.

Controller Layer: Processes requests, applies rules, communicates with models/services and returns JSON responses.

View/Interface Layer: Since this is a headless backend service, the interface is provided through the API routing layer instead of traditional web templates.

14. Project Setup

1. Create the project folder.

2. Create a src folder inside it.

3. Open the project in Visual Studio Code.

4. Initialize the Node.js project.

5. Install the required packages.

Main packages:

express, mongoose, dotenv, cors, jsonwebtoken, bcrypt and @google/genai.

15. Backend Modules

Important modules include:

- server.js – starts the application and database connection.
- app.js – configures the Express application.
- authMiddleware.js – validates authentication tokens.
- errorMiddleware.js – handles errors centrally.
- User.js – defines the user schema.
- FAQ.js – defines the FAQ schema.
- authRoutes.js – handles authentication routes.
- faqRoutes.js – handles FAQ operations.
- aiRoutes.js – handles AI FAQ generation.

16. API Endpoints

Authentication:

POST /api/auth/register – Register a new user.

POST /api/auth/login – Login and receive authentication information.

FAQ:

POST /api/faqs – Create an FAQ (protected).

GET /api/faqs/search?q=configure – Search FAQs (public).

AI:

POST /api/ai/generate-faq – Generate an FAQ using AI (protected).

17. Sample API Operations

Registration request:

```
{

"name": "Priya Sharma",

"email": "priya@writeflow.com",

"password": "securepassword123"

}
```

FAQ creation:

```
{

"question": "How do I configure custom category tags?",
```

"answer": "Navigate to settings panel, append keyword tags arrays, and save schema.",

"category": "Configuration"

}

AI generation:

{

"topic": "Mongoose schema indexing validation runtime workflow optimization"

}

18. Project Execution and Testing

The backend can be started in the development environment using:

```
npm run dev
```

Postman or ThunderClient can be used to test the API endpoints. Testing should verify registration, login, protected

FAQ operations, search operations and AI FAQ generation.

19. Expected Outcome

The system reduces manual FAQ preparation and improves information organization. It maintains structured data
using schema validation, protects private operations using authentication, and supports faster information retrieval through search and indexing. AI integration helps generate FAQ content efficiently.

20. Conclusion

The AI FAQ Assistant provides a secure and intelligent backend solution for FAQ and customer-support automation.

By combining Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt and Google Gemini AI, the system supports

FAQ management, search, authentication and AI-assisted content generation in a modular architecture.

21. Reference Links

API Testing Video:

https://drive.google.com/file/d/1u_w1sigiOynhmqTDbj4NBm3CLlMunEWN/view?usp=sharing

Demo Link:

https://drive.google.com/file/d/1ajAZGgOaowPdyW2x5otcsLI84NeomSY/view?usp=drive_link

CodeDrive: CODE